Agent Small Harness - Open Source Readiness Audit

EXECUTIVE SUMMARY

We performed a thorough open-source readiness audit of the repository. Key observations include: **Legal** - the project needs an explicit open-source license. Without a license, default copyright applies and **others cannot legally use or distribute the code**. We recommend adding a standard permissive license (e.g. MIT or Apache 2.0) immediately. **Dependency Compliance** - we must verify that all third-party libraries are compatible and properly licensed. Tools like *licensecheck* can automate this (it "outputs the licenses used by dependencies and checks if these are compatible with the project license"). **Security** - scan for sensitive data. Use TruffleHog (e.g. ‘trufflehog git file://.‘) to search entire Git history for leaked keys, and GitGuardian’s ggshield (‘ggshield scan path -r .‘) to detect hardcoded secrets. **Vulnerabilities** - run ‘pip-audit‘ on ‘requirements.txt‘ or in a venv to catch known CVEs. Also run Bandit (‘bandit -r .‘) to flag insecure Python patterns. **Testing & CI** - ensure a suite of unit and integration tests exists, with good coverage and an automated pipeline. Best practices call for "Unit Tests ... Integration Tests ... Test Coverage ... Continuous Integration (CI)". We will verify that tests pass (e.g. via ‘pytest‘) and add CI configuration if missing. **Documentation** - include a comprehensive ‘README.md‘ with project overview, install and usage instructions, examples and acknowledgments. Guidelines advise adding a README and a LICENSE file in the repo root. We will also check for CONTRIBUTING, CHANGELOG, CODE_OF_CONDUCT, etc. for community readiness. **Privacy/Export** - if the code processes any personal data or uses cryptography, we will assess GDPR/compliance and US export rules. Notably, open-source projects using standard encryption need no special export notices. **Ethics/Safety** - since this is a model-driven tool, watch for bias or hallucinations in outputs. LLMs are inherently nondeterministic and can produce unexpected results. Users should handle any AI output carefully and include disclaimers.

Below is a checklist of readiness items, a remediation plan, and detailed recommendations.

READINESS CHECKLIST

Category	Check	Status (Pass/Fail)	Notes/Sources
License	LICENSE file present (OSI-approved)	[FAIL] Fail if missing	"without a license... no one may reproduce or distribute"; add MIT/Apache.
License Compliance	Dependency licenses compatible	[PENDING] Pending	Use ‘licensecheck‘ or ‘pip-licenses‘; ensure no GPLv3/AGPL if not intended.
Secrets/API Keys	No secrets or keys in code/history	[PENDING] Pending	Scan with TruffleHog ‘trufflehog git ...‘ and ggshield.
Dependencies	All deps listed (requirements)	[PENDING] Pending	Provide ‘requirements.txt‘ or ‘pyproject.toml‘; pin versions for reproducibility.
Vulnerabilities	Known CVEs in dependencies	[PENDING] Pending	Run ‘pip-audit‘ to catch CVEs; fix or upgrade.
Static Analysis	Code passes security linter (Bandit)	[PENDING] Pending	Run ‘bandit -r .‘ to find common issues.
Tests	Unit & integration tests exist	[PENDING] Pending	Write tests covering core functions (unit) and end-to-end flows (integration).
Test Coverage	Coverage reported (e.g. via Coverage)	[PENDING] Pending	Aim for high coverage and include badge; measure with Coverage.py.
CI Pipeline	CI configured & passing	[FAIL] Fail if none or

broken | Use GitHub Actions or similar; see sample YAML below. |
| **README** | Clear project README | [PENDING] Pending
| Include overview, install, usage, examples. |
| **Other Docs** | Contributing, Code of Conduct, Changelog | [PENDING] Pending
| Add ‘CONTRIBUTING.md‘, ‘CODE_OF_CONDUCT.md‘, ‘CHANGELOG.md‘ for
community. |
| **Privacy** | No hardcoded PII or telemetry | [PENDING] Pending
| If collecting user data, ensure anonymization; if none, note "no
PII handled." |
| **Export Compliance**| No restricted crypto used | [PASS] Pass (likely)
| Standard crypto is fine (no special export steps needed). |
| **Ethics/Safety** | AI output use guidelines in place | [PENDING] Pending
| Consider adding disclaimers; ensure outputs aren’t misused. |

Items marked [FAIL] Fail or [PENDING] Pending require attention before release.

PRIORITIZED REMEDIATION STEPS

1. **Add a License (High priority)**: Include an OSI-approved license file (‘LICENSE‘ or ‘LICENSE.md‘) at the repo root. If unsure, MIT or Apache 2.0 are good permissive choices. Update ‘setup.py/pyproject.toml‘ to declare the license metadata.
2. **Remove Secrets (Critical)**: Use TruffleHog (‘trufflehog git file://.‘) and GitGuardian’s ggshield (‘ggshield scan path -r .‘) to find any embedded secrets. If keys or passwords are found, delete them and rotate them. Add patterns to ‘.gitignore‘ and ‘ggshield‘ ignore if needed. Consider a repo rewrite (‘git filter-branch‘ or BFG) if history contains any leaked secrets.
3. **Lock Down Dependencies (High)**: Pin all Python dependencies with exact versions (and hashes if possible). Audit licenses: run ‘licensecheck‘ or ‘pip-licenses‘ to list all dependency licenses. Ensure no prohibited licenses (e.g. GPL if project intended permissive). If any patent-encumbered libs are included, note them or replace them. For example:
   ‘‘‘bash
   pip install licensecheck
   licensecheck
   ‘‘‘
   This will list each package and highlight incompatibilities. Resolve any conflicts (e.g. remove or replace libraries under non-permissive terms).
4. **Fix Vulnerabilities (High)**: Run ‘pip-audit‘ against your environment or ‘requirements.txt‘:
   ‘‘‘bash
   pip install pip-audit
   pip-audit -r requirements.txt
   ‘‘‘
   For any reported CVEs, upgrade to fixed versions or apply patches. Consider using ‘pip-audit --fix‘ if appropriate. Also run Bandit to spot insecure code patterns:
   ‘‘‘bash
   pip install bandit
   bandit -r .
   ‘‘‘
   Review and address all medium/high findings.
5. **Implement Testing & CI (Medium-High)**: Ensure a comprehensive test suite (unit, integration). Add tests for core functionality; confirm they pass locally (‘pytest‘). Add a CI workflow (e.g. GitHub Actions) to run linting, Bandit, pip-audit, and tests on every push. See sample CI YAML below.
6. **Improve Documentation (Medium)**: Flesh out ‘README.md‘ with sections: project description, installation, usage examples, and dependencies. Add a ‘CHANGELOG.md‘ (using Keep a Changelog style) and ‘CONTRIBUTING.md‘ if accepting contributions. A ‘CITATION.cff‘

can be added if you expect academic citations.
7. **Add Code Quality Checks (Low)**: Integrate linters (e.g. `ruff`, `flake8`). Ensure clean code style. Possibly configure pre-commit hooks for formatting and secrets scanning.
8. **Final Audit (Low)**: Run the final checks (see Smoke Tests below) to verify nothing is missed.

SMOKE TESTS TO INCLUDE BEFORE RELEASE

Before making the project public, automate the following quick checks (examples shown):

- **License Presence**:
  ```bash
  test -f LICENSE || echo "LICENSE file missing!"
  ```
  Ensure `LICENSE` exists and is correct.

- **Secrets Scan**:
  ```bash
  pip install trufflehog ggshield
  trufflehog git file://$(pwd)
  ggshield scan path -r .
  ```
  Ensure no output indicates a leaked secret. GGShield requires setting `GITGUARDIAN_API_KEY` in env.

- **Vulnerability Scan**:
  ```bash
  pip install pip-audit
  pip-audit -r requirements.txt --ignore-vuln GHSA-*
  ```
  Confirm no unresolved vulnerabilities.

- **License Audit**:
  ```bash
  licensecheck .
  ```
  Check that all detected licenses (2-clause BSD, MIT, Apache, etc.) are acceptable.
  The output will mark any incompatibilities.

- **Static Analysis**:
  ```bash
  bandit -r .
  ```
  Verify there are no new critical issues.

- **Test Suite**:
  ```bash
  pytest --maxfail=1 --disable-warnings -q
  ```
  All tests should pass.

- **CI Dry Run**: Use local tools (Act, Docker image) to simulate CI (optional).

Each of these can be scripted as part of a `make verify` or CI job.

RECOMMENDED LICENSE

If none is currently defined, we recommend an open-source license. GitHub strongly advises

that "for your repository to truly be open source, you'll need to license it". Common choices: **MIT** (very permissive, minimal restrictions) or **Apache 2.0** (permissive + explicit patent grant). Both are OSI-approved and compatible with most dependencies. Add the chosen license text to 'LICENSE' and reference it in your metadata (e.g. 'setup.py').

RELEASE CHECKLIST

Before tagging a release, ensure the following:
- **Versioning**: Bump version in code (e.g. '__version__').
- **Changelog**: Update 'CHANGELOG.md' with user-facing changes since last release.
- **Commit Hygiene**: Ensure all changes are committed and PRs reviewed. Optionally sign tags/commits.
- **CI Status**: Confirm CI is green (build, test, lint, security scans).
- **Package Build**: Verify packaging works ('python -m build') and artifacts install ('pip install dist/*.whl').
- **Security Scan**: Re-run 'pip-audit', 'bandit', and 'licensecheck' on the final state.
- **Documentation**: Ensure 'README.md' is current. Include any license badges or CI status badges.
- **Tag & Publish**: Create a Git tag (e.g. 'vX.Y.Z'). On push, GitHub Actions should trigger publishing.
- **Post-Release**: Update docs site (if any) and announce the release.

```mermaid
flowchart LR
    A[Developer Commits Code] --> B[CI: Lint & Static Analysis]
    B --> C[CI: Run Tests (unit/integration)]
    C --> D[CI: Security Scan (pip-audit, bandit, trufflehog)]
    D --> E[CI: License Check (licensecheck)]
    E --> F[CI: Build Package]
    F --> G[Tag Release (Manual/CI trigger)]
    G --> H[CI: Publish to PyPI/GitHub Packages]
    G --> I[GitHub Release notes & Changelog]
```

```yaml
.GITHUB/WORKFLOWS/PUBLISH.YML (SAMPLE)

name: Publish Python package
on:
  push:
    tags:
      - 'v*'
jobs:
  publish:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up Python
      uses: actions/setup-python@v4
      with:
      python-version: '3.11'
      - name: Install build tools
      run: |
      python -m pip install --upgrade pip build wheel twine
      - name: Build package
      run: python -m build --sdist --wheel --outdir dist
      - name: Publish to PyPI
      uses: pypa/gh-action-pypi-publish@v1
```

```
      with:
      user: __token__
      password: ${{ secrets.PYPI_API_TOKEN }}
      - name: Publish to GitHub Packages
      run: |
      twine upload --repository github dist/*
      env:
      TWINE_USERNAME: ${{ github.actor }}
      TWINE_PASSWORD: ${{ secrets.GITHUB_TOKEN }}
'''

RISK MATRIX


| Issue                          | Severity | Remediation                       | Effort |
Time to Fix |
|--------------------------------|:--------:|-----------------------------------|:------:|:--
---------:|
| **Missing license file**       | High     | Add LICENSE (e.g. MIT)            | Low    | 15
min     |
| **Secrets in git history**     | Critical | Remove secrets, rotate keys       | Med    |
1-2 hours  |
| **Vulnerable dep(s)**         | High     | Upgrade/pin dependencies         | Med    | 1-2
hours  |
| **Incompatible licenses**      | Med      | Remove/replace offending libs     | Med    | 1
hour     |
| **No tests/CI**                | High     | Write tests, add CI config        | High   |
1-2 days   |
| **Poor documentation**         | Med      | Expand README, add docs           | Med    |
2-4 hours  |
| **Unpinned deps / no hashes**  | Med      | Pin versions, add hashes          | Low    | 1
hour     |
| **No coverage reporting**      | Low      | Integrate Coverage.py/Codecov     | Low    | 30
min     |
| **Potential PII handling**     | Low      | Ensure no personal data stored    | Low    | 30
min     |
| **Export compliance (crypto)** | Low      | Verify usage of standard crypto   | Low    | 10
min     |
| **AI output safety**           | Med      | Add usage guidelines/disclaimers | Med    | 1
hour     |

*(Severity: Critical/High/Med/Low; Effort: Low/Med/High) - details above.*

LOCAL AUDIT COMMANDS


For each category, we recommend these commands:

- **Search for secrets/keys**:
  '''bash
  # Common patterns
  git grep -niE "password|token|secret|aws_|apikey"
  # Use TruffleHog on entire history
  trufflehog git file://$(pwd)
  # GitGuardian (requires API key)
  ggshield scan path -r .
  '''
```

- **Dependency vulnerabilities**:
  ‘‘‘bash
  pip install pip-audit
  pip-audit -r requirements.txt
  ‘‘‘
  (or simply ‘pip-audit‘ in an activated venv).

- **License analysis**:
  ‘‘‘bash
  pip install licensecheck
  licensecheck .
  ‘‘‘
  This lists each dependency license and flags incompatibilities.

- **Static code analysis**:
  ‘‘‘bash
  pip install bandit
  bandit -r .
  ‘‘‘
  Bandit "finds common security issues in Python code".

- **General sanity checks**:
  ‘‘‘bash
  pytest --maxfail=1 -q   # run test suite
  pip install pip-licenses
  pip-licenses   # list installed package licenses
  ‘‘‘

Each of these should run without errors or warnings before publishing.

REFERENCES

- GitHub: *Licensing a repository* - "If you’re creating an open source project, we strongly encourage you to include an open source license".
- LicenseCheck (pip): "Output the licences used by dependencies and check if these are compatible with the project license".
- TruffleHog: recommends ‘trufflehog git file://‘ to scan Git history for secrets.
- GitGuardian ggshield: "detect more than 200 types of secrets" in code.
- pip-audit: "a tool for scanning Python environments for packages with known vulnerabilities".
- Bandit (PyCQA): "designed to find common security issues in Python code".
- Imageomics Code Checklist: recommends adding a License and detailed README, and having unit/integration tests with CI and coverage.
- Linux Foundation - Export Controls: "As of 2021, if an open source project uses standard cryptography, there are no additional requirements".
- Patel et al. (Generative AI release checklist): highlights that LLM-based products require special ethical/safety evaluation and can be non-deterministic.

All recommendations above follow best practices and should be verified with the cited sources.